

Plano para colocar multiempresa com multibancos

Data: 05/05/2026

Produto: Facilicita.IA — automação de licitações governamentais

Decisão arquitetural prévia: banco-por-cliente (database-per-tenant) — ver memória `project_arquitetura_multi_tenant.md`

Objetivo: documentar como direcionar cada cliente para o banco MySQL correto mantendo uma única URL de produto (`facilicita.com.br`).

1. Recomendação: Subdomínio por cliente

URL personalizada por cliente: `<cliente>.facilicita.com.br`

Exemplos:

- `biohosp.facilicita.com.br` → banco `editais_biohosp`
- `vitasense.facilicita.com.br` → banco `editais_vitasense`
- `arnaldo.facilicita.com.br` → banco `editaisvalida`
- `demo.facilicita.com.br` → banco `editais`

2. Por que subdomínio é a melhor opção

#	Benefício	Detalhe
1	Padrão de mercado SaaS	Atlassian (<code>empresa.atlassian.net</code>), Slack (<code>empresa.slack.com</code>), Notion, Linear, Salesforce, Zendesk — todos usam subdomínio. Comunica profissionalismo automaticamente
2	Isolamento de cookie nativo	Cookies de <code>biohosp.facilicita.com.br</code> não vazam para <code>vitaseense.facilicita.com.br</code> . Sem isso, qualquer XSS em um cliente afeta os outros — é segurança "de graça" pelo browser
3	Sem banco mestre / sem ponto único de falha	Subdomínio resolve o roteamento direto no middleware do backend. Para 50 clientes amanhã o código é o mesmo
4	Login simples	Cliente memoriza uma URL personalizada e loga com email/senha normal. Sem tela de "selecione cliente", sem ambiguidade
5	SEO / branding	Cada cliente tem URL própria para cartão de visita, assinatura de e-mail, integrações webhook. "Facilicita da Bio-Hosp = <code>biohosp.facilicita.com.br</code> " comunica posse
6	Migração futura para domínio próprio é fácil	Quando algum cliente quiser <code>portal.biohosp.com.br</code> , basta CNAME + whitelist de hosts. Não muda nada no código

3. Alternativas consideradas (e por que foram descartadas)

Opção	Problema
Email do login decide o banco	Banco mestre vira ponto único de falha. Email com domínio comum (gmail.com) precisa tela "qual empresa?". Complica recuperação de senha
Tela de seleção pós-login	Login lento (busca user em N bancos). UX adicional. Padrão raro em SaaS B2B moderno
Path-based (<code>facilicita.com.br/biohosp/...</code>)	Cookies não isolam por path. Conflito com rotas reais (<code>/dashboard</code>). URL feia. Quase ninguém usa em SaaS

4. Arquitetura — 4 partes

Parte 1 — DNS wildcard

Configurar 1 entrada DNS:

```
*.facilita.com.br      A      <IP do servidor>
```

Resolve **todos** os subdomínios futuros sem mexer em mais nada. Quando aparecer cliente novo, basta criar o banco — DNS já cobre.

Parte 2 — HTTPS wildcard

Certificado wildcard cobrindo todos os subdomínios:

```
certbot certonly --dns-cloudflare \
-d facilita.com.br \
-d '*.facilita.com.br'
```

Let's Encrypt + cert-manager renova automaticamente a cada 60 dias.

Parte 3 — Middleware Flask que extrai subdomínio

Ler `request.host` antes de cada request, extrair o subdomínio, guardar no contexto. Toda query subsequente usa esse contexto para decidir o banco.

Parte 4 — Pool de engines SQLAlchemy (1 por cliente, lazy)

Em vez de 1 engine global apontando para `editais`, um **dicionário de engines** keyed por subdomínio. Primeira request de `biohosp` cria a engine; próximas reusam (connection pool por banco).

5. Implementação — código concreto

`backend/db_router.py` (novo arquivo, ~50 linhas)

```
"""Roteamento de banco multi-tenant por subdomínio.
```

```
Como funciona:
```

1. Cada request chega com Host header (ex: 'biohosp.facilicita.com.br')
2. get_subdomain() extrai 'biohosp'
3. CLIENTES mapeia 'biohosp' -> 'editais_biohosp'
4. get_engine() retorna engine SQLAlchemy do banco do cliente (cache em memória)
5. get_session() devolve Session conectada ao banco correto

```
Em dev (localhost) cai no banco 'editais' por padrão.
```

```
"""
```

```
from flask import g, request, abort
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from urllib.parse import quote_plus
```

```
# Config base do MySQL
```

```
DB_HOST = "camerascasas.no-ip.info"
```

```
DB_PORT = 3308
```

```
DB_USER = "producao"
```

```
DB_PASS = "112358123"
```

```
DB_BASE_URI = f"mysql+mysqlconnector://{DB_USER}:{quote_plus(DB_PASS)}@{DB_HOST}:{DB_PORT}"
```

```
# Cache de engines e sessions: {db_name: Engine | sessionmaker}
```

```
_engines = {}
```

```
_sessions = {}
```

```
# Lista oficial de clientes
```

```
# Depois pode virar tabela no banco mestre (editais_master.clientes)
```

```
CLIENTES = {
```

```
    "biohosp": "editais_biohosp",
```

```
    "vitasense": "editais_vitasense",
```

```
    "arnaldo": "editaisvalida",
```

```
    "demo": "editais",
```

```
}
```

```
def get_subdomain() -> str:
```

```
    """Extraí subdomínio do Host (biohosp.facilicita.com.br -> biohosp).
```

```
    Em dev (localhost ou IP) cai em 'demo' que aponta pra banco 'editais'.
```

```
    """
```

```
    host = request.host.split(":")[0] # remove porta
```

```
    if host in ("localhost", "127.0.0.1") or host.replace(".", "").isdigit():
```

```
        return "demo"
```

```
    parts = host.split(".")
```

```
    return parts[0] if len(parts) >= 3 else "demo"
```

```
def get_db_name() -> str:
```

```
    sub = get_subdomain()
```

```
    if sub not in CLIENTES:
```

```
        abort(404, f"Cliente '{sub}' não encontrado")
```

```
    return CLIENTES[sub]
```

```
def get_engine():
```

```
    """Retorna engine SQLAlchemy do banco do cliente (cria lazy)."""
```

```
    db = get_db_name()
```

```
    if db not in _engines:
```

```
        _engines[db] = create_engine(
```

```
            f"{DB_BASE_URI}/{db}?charset=utf8mb4",
```

```
            pool_pre_ping=True,
```

```
            pool_recycle=3600,
```

```
            pool_size=5,
```

```
        )
```

```
        _sessions[db] = sessionmaker(bind=_engines[db])
```

```

    return _engines[db]

def get_session():
    """Retorna Session conectada ao banco do cliente."""
    db = get_db_name()
    get_engine() # garante criação
    return _sessions[db]()

```

backend/app.py — substituir `get_db()` global

```

# Antes (linha ~XX):
# def get_db():
#     return SessionLocal()

# Depois:
from db_router import get_session

def get_db():
    return get_session()

```

frontend/vite.config.ts

Já está correto (`allowedHosts: true` ou array com `.facilicita.com.br`). Em dev local, basta adicionar `.localhost` no whitelist:

```

server: {
  host: '0.0.0.0',
  port: 5180,
  cors: true,
  allowedHosts: ['.facilicita.com.br', '.localhost', 'localhost', '127.0.0.1'],
  proxy: {
    '/api': 'http://localhost:5007',
    '/uploads': 'http://localhost:5007',
  },
}

```

Nginx — proxy reverso wildcard

Arquivo `/etc/nginx/sites-available/facilicita.conf`:

```
server {
    listen 443 ssl http2;
    server_name *.facilicita.com.br facilicita.com.br;

    ssl_certificate      /etc/letsencrypt/live/facilicita.com.br/fullchain.pem;
    ssl_certificate_key  /etc/letsencrypt/live/facilicita.com.br/privkey.pem;

    # Backend Flask
    location /api {
        proxy_pass http://localhost:5007;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
    location /uploads {
        proxy_pass http://localhost:5007;
        proxy_set_header Host $host;
    }

    # Frontend Vite (em prod, build estático servido aqui mesmo)
    location / {
        proxy_pass http://localhost:5180;
        proxy_set_header Host $host;
        # WebSocket pro HMR Vite (só em dev)
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
    }
}

# Redireciona HTTP -> HTTPS
server {
    listen 80;
    server_name *.facilicita.com.br facilicita.com.br;
    return 301 https://$host$request_uri;
}
```

6. Etapas para colocar em produção

#	Etapas	Tempo estimado
1	Comprar domínio <code>facilicita.com.br</code> (registro.br ou outro)	15 min
2	Configurar DNS wildcard (<code>*.facilicita.com.br</code> → IP público)	30 min
3	Instalar Nginx na máquina servidor	30 min
4	Gerar certificado wildcard (Let's Encrypt + DNS challenge)	1 hora
5	Configurar <code>nginx.conf</code> com proxy reverso (config acima)	30 min
6	Aplicar <code>db_router.py</code> no backend	30 min
7	Substituir <code>get_db()</code> em <code>app.py</code> para usar <code>get_session()</code> do router	15 min
8	Testar com 2 clientes (criar <code>editais_biohosp</code> clonado do template)	1 hora
9	Atualizar <code>vite.config.ts</code> com <code>allowedHosts</code> se necessário	5 min
10	Substituir links hardcoded <code>localhost:5007</code> no código (se houver)	30 min

Tempo total: ~5 horas de trabalho focado.

7. Onboarding de novo cliente (depois que infra estiver pronta)

Toda vez que um cliente novo assina o produto:

```
# 1. Cria o banco do cliente clonando do template
DB_NAME=$(echo $CLIENTE | tr '[:upper:]' '[:lower:]')
mysql -h camerascasas.no-ip.info -P 3308 -u producao -p \
  -e "CREATE DATABASE $DB_NAME CHARACTER SET utf8mb4"
mysqldump -h ... -P ... -u ... editais_template | \
  mysql -h ... -P ... -u ... $DB_NAME

# 2. Adiciona linha em CLIENTES no db_router.py:
#   "novocliente": "editais_novocliente"

# 3. Cria user admin do cliente no banco dele:
mysql ... $DB_NAME -e "INSERT INTO users (...) VALUES (...)"

# 4. Restart backend pra recarregar dicionário CLIENTES
#   (ou no futuro: tabela master e leitura dinâmica sem restart)

# 5. Pronto. Cliente acessa: https://novocliente.facilicita.com.br
```

Automatização futura: script `scripts/onboard_cliente.sh <nome>` que faz tudo isso em 1 comando.

8. Migrations (atualização de schema em todos os bancos)

Quando mexer em schema (nova coluna, nova tabela, etc.), aplicar em todos os bancos:

```
#!/bin/bash
# scripts/migrate_all.sh
MIGRATION=$1
for db in $(mysql -h ... -u ... -e "SHOW DATABASES LIKE 'editais%'" -N); do
    echo "Migrando $db..."
    mysql -h ... -u ... $db < migrations/$MIGRATION
done
```

Estratégia: sempre que sair feature nova com mudança de schema:

1. Migration roda **primeiro no template** (`editais_template`)
2. Migration roda **em todos os bancos cliente** (script acima)
3. Deploy do código novo

Cuidado: rollback de migration em N bancos é mais delicado. Migrations devem ser sempre **forward-compatible** (código velho aceita schema novo, ex: nova coluna nullable).

9. Banco mestre (opcional, futuro)

Quando o número de clientes crescer (~10+), substituir o dicionário hardcoded `CLIENTES` em `db_router.py` por uma tabela em banco mestre:

```
-- editais_master.clientes
CREATE TABLE clientes (
    id VARCHAR(36) PRIMARY KEY,
    subdominio VARCHAR(50) UNIQUE NOT NULL,
    nome_banco VARCHAR(100) NOT NULL,
    razao_social VARCHAR(255),
    cnpj VARCHAR(18),
    plano ENUM('basic', 'pro', 'enterprise'),
    ativo TINYINT(1) DEFAULT 1,
    criado_em DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

`db_router.py` consulta essa tabela em vez do dicionário. Permite:

- Suspendar cliente sem deploy (`ativo=0`)
- Trocar de banco sem deploy (caso migre cliente VIP pra instância dedicada)
- Auditoria de acessos (quem acessou qual cliente quando)

10. Plano de testes (Definition of Done)

Antes de considerar pronto:

- [] DNS wildcard responde para `xyz.facilicita.com.br` (qualquer subdomínio)
- [] HTTPS wildcard funciona (cert válido)
- [] `biohosp.facilicita.com.br/api/auth/login` autentica em `editais_biohosp`
- [] `arnaldo.facilicita.com.br/api/auth/login` autentica em `editaisvalida`
- [] Sessão de `biohosp` não vaza pra `vitassense` (cookie isolado)
- [] Listar editais em `biohosp` mostra só editais do cliente Bio-Hosp (não vê do Vita-Sense)
- [] Onboarding de cliente novo (clone template + adicionar em CLIENTES + restart) funciona em < 5 min
- [] Subdomínio inexistente retorna 404 limpo (sem traceback)

- [] Migration aplicada via script `migrate_all.sh` afeta todos os bancos
- [] Logs identificam claramente qual cliente fez cada request

11. Custo de implementação resumido

Item	Custo
Código	~80 linhas (db_router.py + ajustes em app.py)
Tempo de dev	~30 min código + 4-5h infra (DNS/HTTPS/Nginx)
Migração de dados	Zero — bancos existentes (<code>editais</code> , <code>editaisvalida</code>) viram clientes existentes
Custo recorrente	Domínio R\$ 40/ano + 1 instância MySQL (já existe)

12. O que NÃO faremos (escopo fora)

- Banco mestre com tabela `clientes` (postergar até ter 10+ clientes)
 - Subdomínio customizado tipo `portal.biohosp.com.br` (postergar até cliente pedir)
 - Multi-região (réplicas geográficas) — postergar até dor real de latência
 - White-label completo (logo/cor por cliente) — postergar até negociação comercial
 - SSO/SAML por cliente — postergar até cliente enterprise pedir
 - Migração de cliente entre regiões — postergar
-

Documento gerado em 05/05/2026 baseado nas decisões arquiteturais consolidadas em sessão.